DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 A

ALGORITHM ENGINEERING: IMPLEMENT, TEST, DEBUG & OPTIMISE

TASK 1 – WORD WRAP USING DYNAMIC PROGRAMMING

```
#include <stdio.h>

#define N 50

int main() {

    int n, width, len[N], dp[N+1], prev[N+1];

    int i, j, spaces, cost, best;

    printf("Enter number of words and line width: ");

    scanf("%d%d", &n, &width);

    printf("Enter word lengths:\n");

    for(i = 1; i <= n; i++)

        scanf("%d", &len[i]);

    dp[0] = 0;

    for(i = 1; i <= n; i++) {

        dp[i] = 999999;

        for(j = i; j >= 1; j--) {

            spaces = width;

            for(int k = j; k <= i; k++)

                spaces -= len[k];

            spaces -= (i - j);

            if(spaces < 0)
```

```
            break;
        cost = (i == n) ? 0 : spaces * spaces;
        if(dp[j-1] + cost < dp[i]) {
            dp[i] = dp[j-1] + cost;
            prev[i] = j;
        }
    }
}
printf("\nDP States:\n");
for(i = 1; i <= n; i++)
    printf("dp[%d] = %d\n", i, dp[i]);
printf("\nMinimum Penalty = %d\n", dp[n]);
printf("\nFormatted Lines:\n");
int end = n, start;
while(end > 0) {
    start = prev[end];
    for(i = start; i <= end; i++)
        printf("Word%d ", i);
    printf("\n");
    end = start - 1;
}
return 0;
}
```

TASK 2 – FRACTIONAL AND 0/1 KNAPSACK

```
#include <stdio.h>
#define N 10
typedef struct {
    int id, w, v;
    float ratio;
} Item;
void sort(Item a[], int n) {
    int i, j;
    Item t;
    for(i = 0; i < n; i++) {
        for(j = i + 1; j < n; j++) {
            if(a[i].ratio < a[j].ratio) {
                t = a[i];
                a[i] = a[j];
                a[j] = t;
            }
        }
    }
}
int main() {
    Item a[N];
    int n, W, i, j, dp[N+1][101];
    int greedy = 0, capacity;
```

```
float fractional = 0, part;
printf("Enter n and capacity: ");
scanf("%d%d", &n, &W);
if(n < 1 || n > N || W < 0 || W > 100) {
    printf("Invalid input!");
    return 0;
}
for(i = 0; i < n; i++) {
    a[i].id = i + 1;
    printf("Enter weight and value of item %d: ", i+1);
    scanf("%d%d", &a[i].w, &a[i].v);
    if(a[i].w <= 0 || a[i].v < 0) {
        printf("Invalid item details!");
        return 0;
    }
    a[i].ratio = (float)a[i].v / a[i].w;
}
sort(a, n);
printf("\nFractional Greedy:\n");
capacity = W;
for(i = 0; i < n && capacity > 0; i++) {
    part = (capacity < a[i].w) ?
          (float)capacity / a[i].w : 1;
    fractional += part * a[i].v;
```

```
    capacity -= part * a[i].w;
    if(part > 0)
        printf("Item %d: %.2f fraction\n", a[i].id, part);
}
printf("Fractional Value = %.2f\n", fractional);
for(i = 0; i <= n; i++)
    for(j = 0; j <= W; j++)
        dp[i][j] = 0;
for(i = 1; i <= n; i++) {
    for(j = 0; j <= W; j++) {
        dp[i][j] = dp[i-1][j];
        if(a[i-1].w <= j &&
           dp[i-1][j-a[i-1].w] + a[i-1].v > dp[i][j])
            dp[i][j] = dp[i-1][j-a[i-1].w] + a[i-1].v;
    }
}
printf("\n0/1 DP Value = %d\n", dp[n][W]);
printf("Selected Items: ");
j = W;
for(i = n; i > 0; i--) {
    if(dp[i][j] != dp[i-1][j]) {
        printf("%d ", a[i-1].id);
        j -= a[i-1].w;
    }
```

```
    }
    printf("\n");
    printf("\nGreedy 0/1 Selection:\n");
    capacity = W;
    for(i = 0; i < n; i++) {
        if(a[i].w <= capacity) {
            capacity -= a[i].w;
            greedy += a[i].v;
            printf("Item %d selected\n", a[i].id);
        }
    }
    printf("Greedy Value = %d\n", greedy);
    return 0;
}
```

TASK 3 – SHORTEST PATH AND GRAPH ALGORITHMS

```
#include <stdio.h>
#define V 8
#define INF 99999
int main() {
    int g[V][V] = {0};
    int dist[V], visited[V] = {0};
    int reach[V][V], d[V][V];
    int u, v, w, i, j, k, count;
    int edges[20][3], m = 0, changed;
```

```
int data[][3] = {
    {0,1,4},{0,2,3},{1,2,1},{1,3,2},
    {2,3,4},{2,4,5},{3,4,2},{3,5,3},
    {4,5,4},{5,6,2},{6,7,3},{4,7,8}
};
m = 12;
for(i = 0; i < m; i++) {
    u = data[i][0];
    v = data[i][1];
    w = data[i][2];
    g[u][v] = g[v][u] = w;
    edges[i][0] = u;
    edges[i][1] = v;
    edges[i][2] = w;
}
printf("DIJKSTRA SHORTEST PATHS FROM A:\n");
for(i = 0; i < V; i++)
    dist[i] = INF;
dist[0] = 0;
for(count = 0; count < V; count++) {
    int min = INF;
    u = -1;
    for(i = 0; i < V; i++) {
        if(!visited[i] && dist[i] < min) {
```

```
            min = dist[i];
            u = i;
         }
      }
      if(u == -1)
         break;
      visited[u] = 1;
      for(v = 0; v < V; v++) {
         if(g[u][v] && !visited[v] &&
            dist[u] + g[u][v] < dist[v])
            dist[v] = dist[u] + g[u][v];
      }
   }
   for(i = 0; i < V; i++)
      printf("%c : %d\n", 'A'+i, dist[i]);
   printf("\nBELLMAN-FORD WITH NEGATIVE EDGE:\n");
   edges[m][0] = 4;
   edges[m][1] = 1;
   edges[m][2] = -3;
   m++;
   for(i = 0; i < V; i++)
      dist[i] = INF;
   dist[0] = 0;
   for(i = 1; i < V; i++) {
```

```
    changed = 0;
    for(j = 0; j < m; j++) {
        u = edges[j][0];
        v = edges[j][1];
        w = edges[j][2];
        if(dist[u] != INF && dist[u] + w < dist[v]) {
            dist[v] = dist[u] + w;
            changed = 1;
        }
    }
    if(!changed)
        break;
}
for(i = 0; i < V; i++)
    printf("%c : %d\n", 'A'+i, dist[i]);
changed = 0;
for(j = 0; j < m; j++) {
    u = edges[j][0];
    v = edges[j][1];
    w = edges[j][2];
    if(dist[u] != INF && dist[u] + w < dist[v])
        changed = 1;
}
printf("Negative cycle: %s\n", changed ? "Detected" : "Not detected");
```

```
printf("\nFLOYD-WARSHALL DISTANCE MATRIX:\n");
for(i = 0; i < V; i++) {
   for(j = 0; j < V; j++) {
      if(i == j)
         d[i][j] = 0;
      else if(g[i][j])
         d[i][j] = g[i][j];
      else
         d[i][j] = INF;
   }
}
d[4][1] = -3;

for(k = 0; k < V; k++)
   for(i = 0; i < V; i++)
      for(j = 0; j < V; j++)
         if(d[i][k] != INF && d[k][j] != INF &&
            d[i][k] + d[k][j] < d[i][j])
             d[i][j] = d[i][k] + d[k][j];
for(i = 0; i < V; i++) {
   for(j = 0; j < V; j++) {
      if(d[i][j] == INF)
         printf("INF ");
      else
```

```
            printf("%3d ", d[i][j]);
        }
        printf("\n");
    }
    printf("\nWARSHALL REACHABILITY MATRIX:\n");
    for(i = 0; i < V; i++) {
        for(j = 0; j < V; j++)
            reach[i][j] = (i == j || g[i][j] != 0);
    }
    for(k = 0; k < V; k++)
        for(i = 0; i < V; i++)
            for(j = 0; j < V; j++)
                reach[i][j] = reach[i][j] ||
                            (reach[i][k] && reach[k][j]);
    for(i = 0; i < V; i++) {
        for(j = 0; j < V; j++)
            printf("%d ", reach[i][j]);
        printf("\n");
    }
    return 0;
}
```

TASK 4 – ASSEMBLY LINE SCHEDULING USING DP

```
#include <stdio.h>

#define N 4

int main() {
    int a1[N] = {7, 9, 3, 4};
    int a2[N] = {8, 5, 6, 4};
    int t1[N] = {0, 2, 2, 2};
    int t2[N] = {0, 3, 3, 3};
    int f1[N], f2[N];
    int p1[N], p2[N];
    int e1 = 2, e2 = 4;
    int x1 = 3, x2 = 2;
    int i, final, line;
    f1[0] = e1 + a1[0];
    f2[0] = e2 + a2[0];
    for(i = 1; i < N; i++) {
        if(f1[i-1] <= f2[i-1] + t2[i]) {
            f1[i] = f1[i-1] + a1[i];
            p1[i] = 1;
        } else {
            f1[i] = f2[i-1] + t2[i] + a1[i];
            p1[i] = 2;
        }
        if(f2[i-1] <= f1[i-1] + t1[i]) {
```

```
        f2[i] = f2[i-1] + a2[i];
        p2[i] = 2;
    } else {
        f2[i] = f1[i-1] + t1[i] + a2[i];
        p2[i] = 1;
    }
    printf("Station %d: Line1=%d Line2=%d\n",
          i+1, f1[i], f2[i]);
}
if(f1[N-1] + x1 <= f2[N-1] + x2) {
    final = f1[N-1] + x1;
    line = 1;
} else {
    final = f2[N-1] + x2;
    line = 2;
}
printf("\nMinimum Time = %d\n", final);
int route[N];
route[N-1] = line;
for(i = N-1; i > 0; i--) {
    if(route[i] == 1)
        route[i-1] = p1[i];
    else
        route[i-1] = p2[i];
```

```
    }
    printf("Optimal Route: ");
    for(i = 0; i < N; i++)
        printf("L%d ", route[i]);
    printf("\n");
    return 0;
}
```

TASK 5 – MINIMUM SPANNING TREE: KRUSKAL, PRIM AND BORUVKA

```
#include <stdio.h>
#define V 6
#define E 9
typedef struct {
    int u, v, w;
} Edge;
Edge edges[E] = {
    {0,1,4},{0,2,3},{1,2,1},
    {1,3,2},{2,3,4},{2,4,5},
    {3,4,2},{3,5,3},{4,5,4}
};
int parent[V];
int find(int x) {
    while(parent[x] != x)
        x = parent[x];
    return x;
```

```
}
int unite(int a, int b) {
    a = find(a);
    b = find(b);
    if(a == b)
        return 0;
    parent[b] = a;
    return 1;
}
void sortEdges() {
    int i, j;
    Edge t;
    for(i = 0; i < E; i++)
        for(j = i+1; j < E; j++)
            if(edges[i].w > edges[j].w) {
                t = edges[i];
                edges[i] = edges[j];
                edges[j] = t;
            }
}
int main() {
    int i, j, cost, count, u, v;
    int g[V][V] = {0};
    int key[V], used[V] = {0};
```

```
printf("KRUSKAL MST:\n");
sortEdges();
for(i = 0; i < V; i++)
   parent[i] = i;
cost = count = 0;
for(i = 0; i < E && count < V-1; i++) {
   if(unite(edges[i].u, edges[i].v)) {
      printf("%c-%c : %d\n",
            'A'+edges[i].u, 'A'+edges[i].v, edges[i].w);
      cost += edges[i].w;
      count++;
   }
}
printf("Kruskal Cost = %d\n", cost);
for(i = 0; i < E; i++) {
   u = edges[i].u;
   v = edges[i].v;
   g[u][v] = g[v][u] = edges[i].w;
}
printf("\nPRIM MST:\n");
for(i = 0; i < V; i++)
   key[i] = 99999;
key[0] = 0;
cost = 0;
```

```
for(i = 0; i < V; i++) {
   int min = 99999;
   u = -1;
   for(j = 0; j < V; j++)
      if(!used[j] && key[j] < min) {
         min = key[j];
         u = j;
      }
   if(u == -1)
      break;
   used[u] = 1;
   cost += key[u];
   for(v = 0; v < V; v++)
      if(g[u][v] && !used[v] && g[u][v] < key[v])
         key[v] = g[u][v];
}
printf("Prim Cost = %d\n", cost);
printf("\nBORUVKA MST:\n");
for(i = 0; i < V; i++)
   parent[i] = i;
cost = count = 0;
while(count < V-1) {
   int cheapest[V];
   for(i = 0; i < V; i++)
```

```
    cheapest[i] = -1;
  for(i = 0; i < E; i++) {
    int a = find(edges[i].u);
    int b = find(edges[i].v);
    if(a == b)
       continue;
    if(cheapest[a] == -1 ||
       edges[i].w < edges[cheapest[a]].w)
        cheapest[a] = i;
    if(cheapest[b] == -1 ||
       edges[i].w < edges[cheapest[b]].w)
        cheapest[b] = i;
  }
  int merged = 0;
  for(i = 0; i < V; i++) {
    j = cheapest[i];
    if(j != -1 &&
       unite(edges[j].u, edges[j].v)) {
        printf("%c-%c : %d\n",
              'A'+edges[j].u,
              'A'+edges[j].v,
              edges[j].w);
        cost += edges[j].w;
        count++;
```

```
            merged = 1;
        }
    }
    if(!merged)
        break;
  }
  printf("Boruvka Cost = %d\n", cost);
  return 0;
}
```

TASK 6 – TRAVELLING SALESMAN PROBLEM USING HELD-KARP DP

```
#include <stdio.h>
#define N 7
#define INF 999999
int main() {
  int cost[N][N] = {
    {0,10,15,20,12,18,14},
    {10,0,35,25,17,22,30},
    {15,35,0,30,12,28,16},
    {20,25,30,0,14,16,24},
    {12,17,12,14,0,20,18},
    {18,22,28,16,20,0,13},
    {14,30,16,24,18,13,0}
  };
  int dp[1<<N][N];
```

```
int parent[1<<N][N];
int mask, j, k, full, best, last;
int i;
for(mask = 0; mask < (1<<N); mask++)
   for(j = 0; j < N; j++) {
      dp[mask][j] = INF;
      parent[mask][j] = -1;
   }
dp[1][0] = 0;
for(mask = 1; mask < (1<<N); mask++) {
   if(!(mask & 1))
      continue;
   for(j = 1; j < N; j++) {
      if(mask & (1<<j)) {
         int prev = mask ^ (1<<j);
         for(k = 0; k < N; k++) {
            if(prev & (1<<k)) {
               if(dp[prev][k] + cost[k][j] < dp[mask][j]) {
                  dp[mask][j] =
                     dp[prev][k] + cost[k][j];
                  parent[mask][j] = k;
               }
            }
         }
```

```
        }
    }
}
full = (1<<N)-1;
best = INF;
last = -1;
for(j = 1; j < N; j++) {
    if(dp[full][j] + cost[j][0] < best) {
        best = dp[full][j] + cost[j][0];
        last = j;
    }
}
printf("Minimum Tour Cost = %d\n", best);
int route[N+1];
mask = full;
for(i = N-1; i >= 1; i--) {
    route[i] = last;
    k = parent[mask][last];
    mask ^= (1<<last);
    last = k;
}
route[0] = 0;
route[N] = 0;
printf("Optimal Route: ");
```

```
    for(i = 0; i <= N; i++)
        printf("%c ", 'A'+route[i]);
    printf("\n");
    return 0;
}
```